PARCIAL 1

jueves, 20 de marzo de 2025 12:08

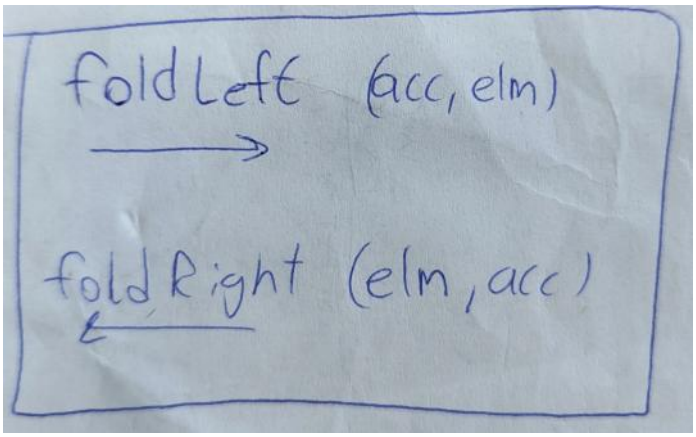
- **toLowerCase** --> lo convierte todo a minúscula
- **replaceAll("n", "m")**--> remplaza la n por m ("\\s" --> es espacio)
- reverse --> invierte la lista
  - Para q nos dé el valor de una posición de la lista invertida:
    - **Reverse.head** -> nos da el primer valor
    - **Reverse(1)** --> cualquier valor menos el primero (nos devuelve el segundo elem de la lista)
- **distinct** --> elimina los valores repetidos de la lista
- **sorted** --> ordena la lista de menor a mayor
- **split(" ")** --> divide una cadena en partes usando un separador (espacios) y devuelve un array
- **flatMap** --> coje cada valor y lo separa para formar una lista
- **toInt** --> convierte un String a un Int
- **toList** --> convierte a una List
- **isEmpty** --> comprueba si la lista está vacía
- **length** / **size** --> nos dice el tamaño
- **splitAt(n)** --> divide una lista en dos partes en la posición n
  - Si queremos coger la primera parte del array q divide el split usamos:. split(" ")(0)
  - La última parte --> split(" ").last

- updated(clave, nuevoValor) --> si ya existe la clave de nuevo valor lo añade y sino crea la clave y luego lo añade
- getOrElse(clave, valorPorDefecto) --> devuelve el valor de la clave y si la clase no existe devuelve el valor por defecto
- .map()--> recorre el mapa como si fuera un for
- .append()--> añade elementos a la lista sin crear otros nuevos
- .mkString(ini, step, fin) --> convierte una lista en un string y comienza por ini, está separado por step y finaliza con fin
- .filter(x=> condición) --> filtrar elementos de una colección
- l1.zip(l2) --> combina las listas l1 y l2 y la convierte en una lista de tuplas
- .groupBy( ) --> crea mapas
  - .groupBy(identity).map { case(p, l) => (p, l.size) --> para saber cuántas veces sale una palabra en una lista
- .startsWith() --> empieza por
- .sortBy() --> ordena una lista
  - .sortBy(_._2) --> ordena el segundo elemento de menor a mayor
  - .sortBy(-_._2) --> ordena el segundo elemento de mayor a menor
- .update(c, v) -> para mapas, añade o cambia el valor del mapa pasándole la clave y el valor

  - ```
    override def equals(obj: Any): Boolean =
      obj match
        case that: ArrayQueue[T] => this.buffer.equals(that.buffer)
        case _ => false
    ```
    ```

🔥 Diferencias clave entre `foldRight` y `foldLeft`

Característica	`foldRight`	`foldLeft`
Orden de evaluación	De **derecha a izquierda**	De **izquierda a derecha**
Asociatividad	`(1 + (2 + (3 + 0)))`	`(((0 + 1) + 2) + 3)`
Uso de `::` en listas	Útil cuando se construyen listas (`::`)	Mejor para reducir (`+`, `*`, etc.)
Eficiencia en listas grandes	Puede ser **más lento** (usa recursión)	Más **eficiente** (usa tail-recursion)

🚀 Reglas clave para recordar

- ✅ `foldRight` usa `(elm, acc)` → Procesa primero el elemento.
- ✅ `foldLeft` usa `(acc, elm)` → Procesa primero el acumulador.